

Data formats and encoding

OLAP (online analytical processing) – designed for complex queries and data analysis. OLAP workloads perform sequential scans on large segments of read-only data. Here, system only needs to find individual tuples to stitch them back together. OLAP refers to software tools used for the analysis of data in business decision-making processes. It allows users to extract and view data from various perspectives. In OLAP, data is represented in columns, with each data store in separate column. Analysis in OLAP are focused, and are using only few columns per table.

OLTP (online transaction processing) – handles large amount of transactional operations in real time, ensuring data consistency and reliability for daily business operations. These workloads are using indexes to find individual tuples without performing sequential scans. They are used for short transactions and can access small amount of data. It manages numerous short atomic operations that keep databases in line.

Storage models:

1. N-ary storage models (NSM)
2. Decompose storage models (DSM)
3. Hybrid storage models (PAX)

n-ary storage models: used in databases where all records are stored together, ie all attributes are stored for a single tuple contiguously in a single page. In this model, the storage is a sequence of N-tuples where N represents number of table columns. This is ideal for OLTP where txns tends to access individual tuples and insert-heavy workloads. It takes only one fetch to be able to get all of the attributes for a single tuple. Page sizes are usually some constant multiples of 4KB hardware pages (Oracle (4KB), Postgres (8KB), MySQL (16KB)). This model is useful for single-tuple access and frequent inserts or updates. Tuples are stored in tables, which are stored in files that are part of the DBMS. Architecture of this storage model is based on pages, where one page is unit of reading or writing. Advantages of this model are fast inserts, update and deletes, and it is good for queries that need the entire tuple. Disadvantage is that it's inefficient for scanning large portions of table and/or subset of the attribute.

Decomposition storage models (DSM): storage model that organizes data by storing each attribute of a record contiguously, making it ideal for OLAP. Each attribute of a record is stored separately, allowing efficient access to specific attributes during read-heavy operations. This model is also known as 'column store' model. Advantages of this storage model are reduced amount of I/O because the DBMS only reads the data that it needs for that query, and better query processing and data compression. Disadvantage is that it is slow for point queries, inserts, updates and deletes because of tuple splitting/stitching.

This storage model decomposes the complete data system into parts. Processing of the system is done separately from the storing. Data is stored either on the disk or on shuffle memory in

between two processes. Processing of tuples takes a lot of time since each tuple is processed separately.

DSM-physical organisation: attributes and meta-data are stored in separate arrays of fixed-length values. Using offset into these arrays, most systems identify unique physical tuples. There is a separate file per attribute, with a dedicated header area for meta-data.

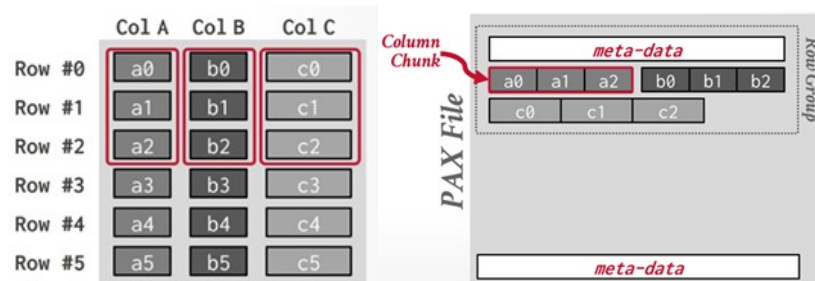


DSM-tuple identification:

1. Fixed-length offset – the value in a given column belongs to the same tuple as the value in another column at the same offset. Therefore, every single value within the column has the same length. To find a specific tuple (to jump to an offset) use a simple arithmetic.
2. Embedded tuple IDs – for every attribute in the columns, the DBMS stores a tuple ID with it. The system also stores a mapping to tell it how to jump to every attribute that has that ID. This method has a large storage overhead because it needs to store a tuple ID for every attribute entry. To find offset for a given ID, we need to use auxiliary data structures.

Hybrid storage models (PAX – partition attribute across): storage model that vertically partitions attributes within the database page. Parquet and ORC use this model. Processing is faster on columnar storage, while retaining the spatial locality benefits of row storage.

PAX – physical organisation: In this model, the rows are horizontally partitioned into row groups. Within each row group, the attributes are vertically partitioned into columns. Each row group is similar to a column store for its subset of the rows. PAX file has a global header containing a directory with offsets to the file's row groups, and each row groups maintains its own header with meta-data about its contents.



Open-source persistent data formats:

1. HDF5 (1998) – multi-dimensional arrays for scientific workloads. The only open source distributed file system
2. Apache Avro (2009) – row-oriented format for Hadoop that replaces SequenceFiles
3. Apache Parquet (2013) – compressed columnar storage from Cloudera/Twitter for Impala
4. Apache ORC (2013) – compressed columnar storage for Meta for Apache Hive
5. Apache CarbonData (2016) – compressed columnar storage with indexes Huawei
6. Apache Arrow (2016) – in-memory compressed columnar storage from Pandas/Dremio, used for processing

Format design decision:

- File meta-data
- Format layout
- Type system
- Encoding schemas
- Block compression
- Filters
- Nested data

File meta-data: files that contain all necessary information to interpret their contents without external data dependencies. These files are self-contained to increase portability. Each file maintains global meta-data about its contents, usually in footer. The content includes: table schema, row group offset/length, tuple count / zone maps. Meta-data includes information about file type, creation date, modification data, author, file size and location.

Format layout: the most common formats use PAX storage model that splits data in row groups that contain one or more column chunks. The size of row group varies per implementation and makes compute / memory trade-offs.

Type system: defines the data type that the format supports. It can be physical and logical. Physical has low level byte representation while logical uses auxiliary types that map to physical types. Different formats vary in the complexity of their type system that determines how much upstream producer / consumer needs to implement.

Encoding schemas: specify how format stores the bytes for contiguous / related data. It includes dictionary encoding, run-length encoding (RLE), bitpacking, delta encoding and frame-of-reference (FOR). Dictionary encoding defines the index of its associated particular value. It is useful for repetitions. In RLE consecutive occurrences of the same data value are stored as a single value and a count. Bitpacking is used on top of dictionary encoding. Delta encoding is a way of storing differences (deltas) between sequential data rather than complete files.

Dictionary compression: represents lossless data compression technique that utilizes a dictionary to encode data. Replaces frequent values with similar fixed-length codes and maintains a mapping (dictionary) from the codes to the original values. Codes represent positions in hash tables or byte offsets. It must handle a case when number of distinct values in a

column chunk is too large. For Parquet, maximal dictionary size is 1Mb. ORC precomputes number of distinct values and disables them if there is too many.

Original Data

name
William
Andrea
Andy
Matt
Andy
Andy
Andy
Andy

Unsorted Dictionary

len	value
6	Andrea
7	William
4	Andy
4	Matt

pos

1
0
2
3
2
2
2
2
2

offset

7
0
13
17
13
13
13
13
13

vs.

Sorted Dictionary

len	value
6	Andrea
4	Andy
4	Matt
7	William

pos

3
0
1
2
1
1
1
1
1

offset

14
0
7
11
7
7
7
7
7

vs.

Dictionary compressions design decisions:

1. Eligible data types (parquet – all data, ORC – strings)
2. Compress encoded data (parquet – RLS+bitpicking, ORC – RLE, delta encoding, bitpicking, FOR)
3. Expose dictionary – (parquet, ORC – move toward supporting)

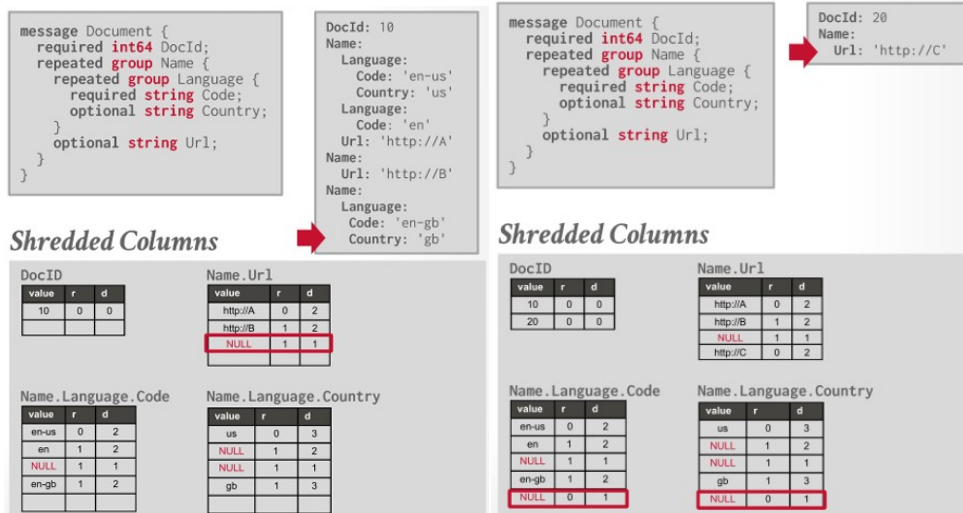
Block compression: compresses data using general-purpose algorithm. Scope of compression is based on the data provided as input (LZO, LZ4, Snappy, Zstd). Considerations about block compressions are computational overhead, compress vs decompress speed, and data opaqueness.

Filters:

1. Zone maps – maintain min/max values per column at the file-level and row group-level. Parquet and ORC store zone maps in the header of each row.
2. Bloom filters – track the existence of value for each column in a row group. They are more effective if values are clustered. They use hash functions. Parquet uses split block bloom filter from Impala.

Nested data:

1. Record shredding – stores paths in nested data structure as separate columns with additional meta-data about paths. There is definition level and repetition level. Definition level – how many optional elements are defined in the path to an attribute. Repetition level – how many times a structure has been repeated.



- Length + presence encoding – stores path in nested structure as separate columns, but maintains additional columns to 1. Track the number of entries at each path level (length); 2. Whether a key exists at that level for a record (presence)

